Lab 1. Decommenter

System Programming Assignment

Soowan Park
Secure & Reliable Systems Lab.

What Is Decommenter

```c
#include <stdio.h>
#include <stdlib.h>

/* Program Author: Muhammad Asim Jamshed */
/* This does simple checks */

/ * This should generate a compilation error */
/
*
This is not a comment
*
/
/ T*his is also not a commen*t/
/* The main function is commented *
/


int main( int argc, char** argv ) {

    printf( "Hello World!\n" );
    return EXIT_SUCCESS;
}
*/
```

```c
#include <stdio.h>
#include <stdlib.h>



/ * This should generate a compilation error */
/
*
This is not a comment
*
/
/ T*his is also not a commen*t/
```

Decommented test0.c

- Decommenter: Tool to remove comments within source code files

Usage Instructions for Decommenter

```
./decomment < somefile.c > output 2> errors
```

- Your executable file
- Target(input) file
- Result
- Error log

- Input and output for “decomment” should be handled via redirection
  - Default functionality of the Linux shell
  - No special additional work required

Requirements – Replace Each Comment with a Space (1/9)

Standard Input Stream	Standard Output Stream	Standard Error Stream
abc/*def*/ghi$_n$	abc$_s$ghi$_n$	

- In the following examples,
  - "$_s$": a space character
  - "$_n$": a newline character

Requirements – Support Both Types of Comments (2/9)

Case	Standard Input Stream	Standard Output Stream	Standard Error Stream
Single line comment	abc//def$_n$	abc$_{sn}$	
Multi-line comment	abc/*def$_n$ghi*/jkl$_n$mno$_n$	abc$_{sn}$jkl$_n$mno$_n$	

- 2 Types of comments:
  - "//": Single line comment
  - "/* … */": Multi-line comment
- Ensure the program **adds blank lines** as needed to **preserve original line numbering**

Requirements – Comments within Comments(3/9)

Standard Input Stream	Standard Output Stream	Standard Error Stream
abc/*def/*ghi*/jkl*/mno$_n$	abc$_s$jkl*/mno$_n$	

- A multi-line comment begins when the preprocess encounters "/*" for the first time
- The comment ends when it encounters "*/" after the comment begins

Requirements – Exceptions: Comment in Str or Char Const (4/9)

Case	Standard Input Stream	Standard Output Stream	Standard Error Stream
Part of the string	abc"def/*ghi*/jkl"mno$_n$	abc"def/*ghi*/jkl"mno$_n$	
Part of the character constant	abc'def/*ghi*/jkl'mno$_n$	abc'def/*ghi*/jkl'mno$_n$	

- Do not de-comment the comment when:
  - Part of the **string or character constant**

Requirements – Handle '\'+ following char as ordinary characters (5/9)

Case	Standard Input Stream	Standard Output Stream	Standard Error Stream
Part of the string	abc"def\"ghi"jkl$_n$	abc"def\"ghi"jkl$_n$	
Part of the character constant	abc'def\'ghi'jkl$_n$	abc'def\'ghi'jkl$_n$	

- Handle '\'+ following char as ordinary characters when
  - Within the string or character constant
- A C compiler would incur an error (multiple characters in a character constant)
  - But many C preprocessors would not
  - And your program should not either

Requirements – Newline Characters in Str or Char Const (6/9)

Case	Standard Input Stream	Standard Output Stream	Standard Error Stream
Part of the string	abc"def$_n$ghi"jkl$_n$	abc"def$_n$ghi"jkl$_n$	
Part of the character constant	abc'def$_n$ghi'jkl$_n$	abc'def$_n$ghi'jkl$_n$	

- Handle the newline character in str or chat const without generating errors or warnings
- A C compiler would incur an error (newline character in a string constant)
  - But many C preprocessors would not
  - And your program should not either

Requirements – Handle Unterminated Str and Char Const (7/9)

Case	Standard Input Stream	Standard Output Stream	Standard Error Stream
Part of the string	abc"def/*ghi*/jkl$_n$	abc"def/*ghi*/jkl$_n$	
Part of the character constant	abc'def/*ghi*/jkl$_n$	abc'def/*ghi*/jkl$_n$	

- Handle unterminated str and char const w/o generating errors or warnings
- A C compiler would incur an error (unterminated str and char constant)
  - But many C preprocessors would not
  - And your program should not either

Requirements – Error: EOF Before a Comment Is Terminated (8/9)

Standard Input Stream	Standard Output Stream	Standard Error Stream
abcdef$_n$ghi/*$_n$	abcdef$_n$ghi$_{sn}$	Error:$_s$line$_s$2:$_s$unterminated$_s$comment$_n$

- Include the line number where the comment starts in the error message
- Return EXIT_FAILURE when an unterminated comment was detected
  - Except for this case, you should return EXIT_SUCCESS in all other cases
  - EXIT_FAILURE and EXIT_SUCCESS are defined as macros in stdlib.h
  - So, add `#include <stdlib.h>` to your C code

Requirements – Miscellaneous Features (9/9)

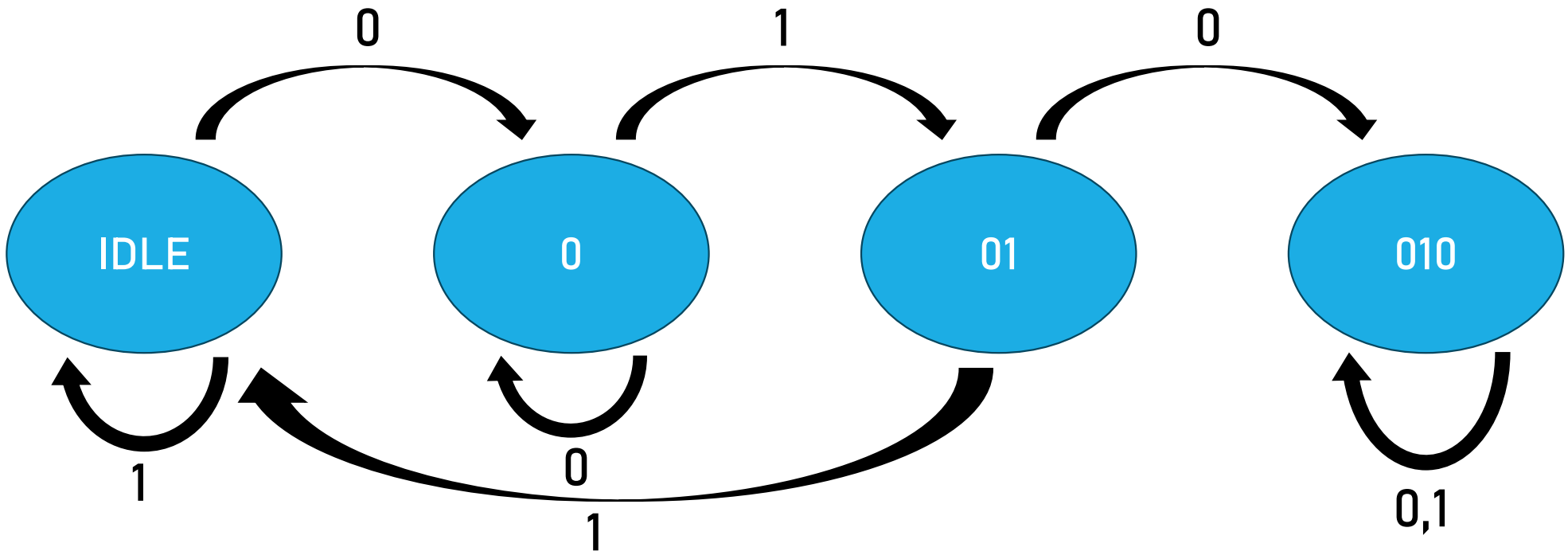
- Your program should work for standard input lines of any length
- You may assume that the final line of the standard input stream ends with a newline character
- Backslash-newline character(\ ) sequence does not occur in the standard input stream
  - Logical lines == physical lines in the standard input stream
- You may place all source code in a single file
- We suggest using standard C getchar() function, fprintf() function
- Etc…
- Please read the **README.md**

Do Your Assignment Step-by-Step

- Step 1. Design a state transition diagram (dfa.pdf or dfa.jpg …) (*handwritten*)
- Step 2. Create source code (decomment.c)
- Step 3. Preprocess, compile, assemble, and link (.i, .s, .o, executable)
- Step 4. Execute
- Step 5. Create a readme file (without filename extension)
- …
- Please refer to the README.md's "Logistics" part!

Deterministic Finite Automaton (or Finite State Machine)

- You've learned it in logic design class
- Your diagram must include both states and transitions
- Ex) '010' Detector



How to Test Your Code

- “assignment1/reference/” directory
  - Sample executable file (sampledecomment) is located
- “assignment1/test_files/” directory
  - Sample test*.c files are located
- “diff” command: Check differences between two files
- Here is an example:
  - ./sampledecomment < test0.c > output1 2> errors1
  - ./decomment < test0.c > output2 2> errors2
  - diff -c output1 output2
  - diff -c errors1 errors2
  - rm output1 errors1 output2 errors2

Submit Format (1/2)

Structure of directory:

```
YourID_assign1
  |-decomment.c (source file)
  |-decomment.i (preprocessed file)
  |-decomment.s (assembly file)
  |-decomment.o (object file)
  |-decomment (executable file)
  |-readme
  `-dfa.pdf or dfa.jpg ...
```

Example:

```
202500000_assign1
  |-decomment.c (source file)
  |-decomment.i (preprocessed file)
  |-decomment.s (assembly file)
  |-decomment.o (object file)
  |-decomment (executable file)
  |-readme
  `-dfa.pdf or dfa.jpg ...
```

- Please set **files and directory's names** to match the examples above
- Please place all source code in a **single file**
- Structure files and directories as shown above, then proceed with **compression**
- **Deadline: ~3.19 23:59**

Submit Format (2/2)

- Helpful commands
  - mkdir 202500000_assign1
  - mv readme dfa.pdf 202500000_assign1
  - mv decomment.c decomment.i decomment.s decomment.o 202500000_assign1
  - mv decomment 202500000_assign1
  - tar zcf 202500000_assign1.tar.gz 202500000_assign1
- Submit '202500000_assign1.tar.gz' to **ETL** submission page

Grading

- Submission format (Penalty for non-compliance)
- From the user's point of view: Correct functioning of the program
- From the programmer's point of view(FYI):
  - A program has quality if it is well styled and thereby easy to maintain
  - Here are the examples:

  - 1. **Names**: Clear and consistent style for variable and function names
  - 2. **Comments**: Each function should begin with a comment that describes what the function does from the point of view of the caller
  - 3. **Function modularity (most important)**: Your program should not consist of one large main function, instead your program should consist of multiple small functions
  - 4. **Line lengths**: Limit line lengths in your source code to 72 characters (for better readability)

FAQ

- Can I modify the provided skeleton code?
  - Yes, you can modify part or all of it.
- Can I write the readme in Korean?
  - Yes.
- Corner case questions
  - Run **sampledecomment** first if you have any doubt. If you suspect it misbehaves, please report it.